

Travaux Pratiques — Régression Linéaire avec TensorFlow/Keras

```
# =====
# RÉGRESSION LINÉAIRE AVEC TENSORFLOW/KERAS
# Université Alger 3 - Master Économie Quantitative
# =====

import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# =====
# EXERCICE 1 : PRISE EN MAIN ET VÉRIFICATION DE L'ENVIRONNEMENT
# =====

print("=" * 60)
print("EXERCICE 1 : ENVIRONNEMENT")
print("=" * 60)

print(f"TensorFlow version : {tf.__version__}")
print(f"Keras version      : {keras.__version__}")

gpu = tf.config.list_physical_devices('GPU')
print(f"GPU disponible      : {gpu}")

print("""
Analyse :

Q1 - Version TensorFlow : déterminée via tf.__version__, attribut
    intégré au module TensorFlow, accessible dès l'import.

Q2 - Version Keras : déterminée via keras.__version__. Keras est
    désormais intégré directement dans TensorFlow (tf.keras),
    donc son numéro de version est lié à celui de TensorFlow.
    On l'obtient en important keras depuis tensorflow puis en
    lisant l'attribut __version__.

Q3 - GPU : tf.config.list_physical_devices('GPU') retourne la liste
    des GPUs détectés par TensorFlow. Une liste vide [] signifie
    qu'aucun GPU n'est disponible → entraînement sur CPU uniquement.

Q4 - Impact de l'absence de GPU : le modèle utilisera le CPU pour
    tous les calculs (multiplication matricielle, rétropropagation).
    Pour un modèle linéaire de très faible dimension (1 entrée,
    1 neurone, 5 exemples), l'impact sur le temps de calcul est
    totalement négligeable. Le GPU n'apporte un gain significatif
    que pour des architectures profondes avec de grands volumes de
    données (millions d'exemples, centaines de couches).
""")

# =====
# PRÉPARATION DES DONNÉES
# =====

surface = np.array([50, 80, 100, 120, 150], dtype=float).reshape(-1, 1)
prix     = np.array([100, 160, 200, 240, 300], dtype=float).reshape(-1, 1)

max_surface = np.max(surface)    # 150
max_prix     = np.max(prix)       # 300
```

```

surface_norm = surface / max_surface
prix_norm     = prix      / max_prix

# =====
# EXERCICE 2 : ANALYSE CRITIQUE DU RÉSULTAT D'ENTRAÎNEMENT
# =====

print("=" * 60)
print("EXERCICE 2 : ANALYSE DE LA MSE INITIALE")
print("=" * 60)

model = keras.Sequential([
    keras.layers.Dense(1, input_shape=(1,), activation='linear')
])

model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
              loss='mse')

history = model.fit(surface_norm, prix_norm,
                    epochs=100,
                    verbose=0,
                    batch_size=len(surface_norm))

mse = history.history['loss'][-1]
rmse = np.sqrt(mse)

print(f"MSE finale = {mse:.6f}")
print(f"RMSE finale = {rmse:.6f}")

print(f"""
Analyse mathématique :
    MSE = (1/n) × Σ (y_pred - y_true)²
    La MSE mesure l'écart quadratique moyen entre prédictions et vérités.
    Le carré pénalise fortement les grandes erreurs.

Les données normalisées sont dans l'intervalle [0, 1].
Une MSE = {mse:.3f} (RMSE ≈ {rmse:.3f}) est anormalement élevée :
l'erreur dépasse l'amplitude même des données → non-convergence.

Valeur acceptable : MSE < 0.01 (RMSE < 0.1) sur données normalisées [0,1].

Hypothèse sur les causes (avant toute modification du programme) :
    Sans toucher au code, deux paramètres du programme initial semblent
    responsables de cette MSE élevée :
    1. learning_rate = 0.001 → probablement trop faible. Un pas de
       gradient aussi petit pourrait ne pas permettre au modèle de
       descendre suffisamment vers le minimum en si peu d'itérations.
    2. epochs = 100 → probablement insuffisant. Même avec un bon LR,
       100 passages sur 5 exemples peut être trop court pour converger.
    Ces deux paramètres combinés expliquent vraisemblablement pourquoi
    le modèle produit des prédictions incohérentes (valeurs négatives).
""")

# =====
# EXERCICE 3 : EXTRACTION DE L'ÉQUATION DU MODÈLE
# =====

print("=" * 60)
print("EXERCICE 3 : ÉQUATION DU MODÈLE (avant correction)")
print("=" * 60)

w_raw, b_raw = model.layers[0].get_weights()
w = float(w_raw[0][0])
b = float(b_raw[0])

```

```

print(f"\nPoids extraits (espace normalisé) :")
print(f"  w = {w:.6f}")
print(f"  b = {b:.6f}")
print(f"\nÉquation normalisée :")
print(f"  prix_norm = {w:.4f} * surface_norm + {b:.4f}")

# Dénormalisation :
#  prix_norm = w * surface_norm + b
#  prix / max_prix = w * (surface / max_surface) + b
#  prix = w * (max_prix / max_surface) * surface + b * max_prix
w_real = w * max_prix / max_surface
b_real = b * max_prix

print(f"\nÉquation réelle (dénormalisée) :")
print(f"  Prix (k€) = {w_real:.4f} * Surface (m²) + {b_real:.4f}")

# Vérification manuelle pour Surface = 100 m²
surface_test = 100.0
surface_test_norm = surface_test / max_surface
prix_test_norm = w * surface_test_norm + b
prix_test_reel = prix_test_norm * max_prix

print(f"\nVérification manuelle pour Surface = 100 m² :")
print(f"  surface_norm = 100 / {max_surface:.0f} = {surface_test_norm:.4f}")
print(f"  prix_norm      = {w:.4f} * {surface_test_norm:.4f} + {b:.4f} = {prix_test_norm:.4f}")
print(f"  Prix réel      = {prix_test_norm:.4f} * {max_prix:.0f} = {prix_test_reel:.2f} k€")
print(f"  (Valeur attendue : 200 k€ – écart dû à la non-convergence)")

# =====
# EXERCICE 4 : DIAGNOSTIC D'UN PROBLÈME DE CONVERGENCE
# =====

print("\n" + "=" * 60)
print("EXERCICE 4 : DIAGNOSTIC")
print("=" * 60)

pred = model.predict(surface_norm, verbose=0)
print("Prédictions (normalisées) :", np.round(pred.flatten(), 4))
print("Vérités terrain           :", prix_norm.flatten())

print(f"""
Analyse des valeurs fournies dans l'énoncé :
Prédictions   : [-0.1458, -0.2918, -0.3892, -0.4866, -0.6326]
Vérités       : [ 0.3333,  0.5333,  0.6667,  0.8000,  1.0000]

- Toutes les prédictions sont NÉGATIVES alors que les prix normalisés
  sont strictement positifs (entre 0.33 et 1.00). Physiquement absurde.
- Les prédictions décroissent quand la surface augmente, alors que
  la relation réelle est croissante → le poids w est négatif (sens inverse).
- L'écart absolu augmente avec la surface : le modèle diverge sur les
  grandes surfaces.

Les deux paramètres du programme initial responsables du dysfonctionnement :

1. learning_rate = 0.001 → trop faible : Adam fait de très petits
  pas dans l'espace des poids. En 100 époques, le modèle n'a pas
  eu le temps de quitter son état initial aléatoire (poids négatif)
  pour converger vers le minimum correct.

2. epochs = 100 → insuffisant : avec un LR de 0.001, il faudrait
  plusieurs milliers d'époques. 100 itérations sont largement
  insuffisantes pour atteindre la convergence.

Relation learning rate (η) ↔ vitesse de convergence :
À chaque époque :  $w \leftarrow w - \eta \times \nabla L$ 

```

- η trop faible $\rightarrow$ pas trop petit $\rightarrow$ convergence très lente ou incomplète.
- η trop élevé $\rightarrow$ pas trop grand $\rightarrow$ oscillations, divergence possible.
- η optimal (ici 0.1) $\rightarrow$ descente régulière et rapide vers le minimum.

Qu'est-ce que la convergence d'un modèle ? Comment la loss la détecte ?

Un modèle converge quand la loss cesse de décroître et se stabilise à une valeur minimale proche de 0. On la détecte en traçant la courbe $loss = f(\text{époques})$: elle doit être décroissante et s'aplatir.

Ici, $loss \approx \{mse:.3f\}$ reste très élevée $\rightarrow$ non-convergence évidente.

```
"""
```

```
# =====
# EXERCICE 5 : CORRECTION ET OPTIMISATION DU PROGRAMME
# =====
```

```
print("=" * 60)
```

```
print("EXERCICE 5 : VERSION CORRIGÉE")
```

```
print("=" * 60)
```

```
model2 = keras.Sequential([
    keras.layers.Dense(1, input_shape=(1,))
])
```

```
model2.compile(optimizer=keras.optimizers.Adam(learning_rate=0.1),
               loss='mse')
```

```
# Entraînement correct en une seule fois (idiomatique et efficace)
```

```
history2 = model2.fit(surface_norm, prix_norm,
                      epochs=500,
                      verbose=0,
                      batch_size=len(surface_norm))
```

```
# Affichage de la loss tous les 100 époques
```

```
loss_history2 = history2.history['loss']
```

```
for i in [99, 199, 299, 399, 499]:
```

```
    print(f"Époque {i+1:>3}  $\rightarrow$  Loss = {loss_history2[i]:.10f}")
```

```
print(f"""
```

```
Évolution de la loss (Époque 1  $\rightarrow$  500) :
```

- Époque 1 : Loss $\approx \{loss_history2[0]:.6f\}$ $\rightarrow$ valeur initiale aléatoire
- Époque 100 : Loss $\approx \{loss_history2[99]:.6f\}$ $\rightarrow$ descente rapide
- Époque 500 : Loss $\approx \{loss_history2[-1]:.10f\}$ $\rightarrow$ convergence quasi-parfaite

La loss chute très rapidement dès les premières époques grâce au learning rate de 0.1, puis se stabilise vers 0 (ajustement parfait sur des données parfaitement linéaires sans bruit).

```
"""
```

```
# Visualisation
```

```
plt.figure(figsize=(8, 4))
```

```
plt.plot(loss_history2, color='steelblue')
```

```
plt.yscale('log')
```

```
plt.title("Convergence de la Loss (MSE) - Exercice 5")
```

```
plt.xlabel("Époques")
```

```
plt.ylabel("MSE (échelle log)")
```

```
plt.grid(True, which='both', linestyle='--', alpha=0.5)
```

```
plt.tight_layout()
```

```
plt.savefig("convergence_loss.png", dpi=150)
```

```
plt.show()
```

```
print("(Graphe sauvegardé : convergence_loss.png)")
```

```
# =====
# EXERCICE 6 : INTERPRÉTATION DES RÉSULTATS FINAUX
# =====
```

```
print("=" * 60)
```

```
print("EXERCICE 6 : INTERPRÉTATION")
```

```

print("=" * 60)

w2_raw, b2_raw = model2.layers[0].get_weights()
w2 = float(w2_raw[0][0])
b2 = float(b2_raw[0])

w_real2 = w2 * max_prix / max_surface
b_real2 = b2 * max_prix

mse_final = loss_history2[-1]

print(f"\nÉquation finale :")
print(f" Prix (k€) = {w_real2:.4f} × Surface (m²) + {b_real2:.4f}")

# Vérification manuelle pour 100 m²
s_norm_test = 100.0 / max_surface
p_norm_test = w2 * s_norm_test + b2
p_reel_test = p_norm_test * max_prix
print(f"\nVérification pour 100 m² :")
print(f" Prix prédit = {p_reel_test:.2f} k€ (attendu : 200 k€)")

print(f"""
Analyse approfondie :
    • MSE finale ≈ {mse_final:.2e} → ajustement quasi-parfait.
      Cela est attendu : les données sont parfaitement linéaires (sans
      bruit), donc un modèle linéaire peut les interpoler exactement.

    • Une MSE ≈ 0 n'est pas toujours souhaitable en pratique :
      sur des données réelles (bruitées), cela signalerait un
      surapprentissage (overfitting) – le modèle mémorise le bruit
      au lieu d'apprendre la tendance générale.

Limites du modèle :
    - Une seule variable explicative (surface), alors qu'un prix
      dépend d'autres facteurs (localisation, étage, état, etc.).
    - Hypothèse de linéarité forte, rarement vérifiée sur le marché réel.
    - Absence de séparation train / test → impossible d'évaluer la
      capacité de généralisation.
    - Données sans bruit → non représentatives d'un cas réel.

Améliorations possibles :
    - Régression multiple avec plusieurs variables explicatives.
    - Séparation train/test (80/20) et validation croisée.
    - Régularisation (L1/L2) pour réduire le surapprentissage.
    - Réseau de neurones non linéaire pour capturer des relations complexes.
""")

# =====
# EXERCICE 7 : SYNTHÈSE ET RÉFLEXION CRITIQUE
# =====

print("=" * 60)
print("EXERCICE 7 : RAPPORT DE SYNTHÈSE")
print("=" * 60)

rapport = """
RAPPORT DE SYNTHÈSE – Régression Linéaire avec TensorFlow/Keras
=====

1. NORMALISATION DES DONNÉES : POURQUOI ET COMMENT
La normalisation consiste à ramener les données dans l'intervalle
[0, 1] en divisant chaque variable par sa valeur maximale :
x_norm = x / x_max. Cette étape est indispensable car les
algorithmes de gradient (comme Adam) sont sensibles à l'échelle
des données : des valeurs élevées (ex. surface en m², prix en k€)
produisent des gradients de grande amplitude qui rendent
l'optimisation instable. La normalisation homogénéise l'espace

```

des paramètres et accélère la convergence.

2. FONCTION DE PERTE MSE : DÉFINITION ET INTERPRÉTATION

La MSE (Mean Squared Error) est définie par :

$$\text{MSE} = (1/n) \times \sum (\hat{y}_i - y_i)^2$$

Elle mesure l'écart quadratique moyen entre les prédictions et les valeurs réelles. Le carré pénalise fortement les grandes erreurs. Dans l'espace normalisé $[0,1]$, une $\text{MSE} < 0.01$ indique une bonne convergence. Une $\text{MSE} \approx 2.317$ (Exercice 2) révèle que le modèle n'a pas convergé : les prédictions sont hors de l'intervalle cible.

3. HYPERPARAMÈTRES : LEARNING RATE ET NOMBRE D'ÉPOQUES

Le learning rate (η) contrôle la taille du pas à chaque mise à jour des poids : $w \leftarrow w - \eta \times \nabla L$. Un η trop faible (0.001) ralentit la convergence ; un η trop élevé (> 1) provoque des oscillations. Le nombre d'époques détermine combien de fois le modèle parcourt les données. Avec $\eta = 0.1$ et 500 époques, le modèle converge vers $\text{MSE} \approx 0$ sur ces données linéaires.

4. EXTRACTION DES COEFFICIENTS D'UN MODÈLE DE RÉGRESSION

La méthode `model.layers[0].get_weights()` retourne (w , b).

Pour obtenir l'équation réelle à partir du modèle normalisé :

$$w_{\text{réel}} = w \times (\text{max_prix} / \text{max_surface})$$

$$b_{\text{réel}} = b \times \text{max_prix}$$

Ce qui donne : $\text{Prix (k€)} = 2.0000 \times \text{Surface (m}^2\text{)} + 0.0000$, retrouvant exactement la relation génératrice des données.

5. INTERPRÉTATION DES PRÉDICTIONS ET ÉVALUATION DES PERFORMANCES

Avant correction ($\text{MSE} = 2.317$), les prédictions étaient négatives, signe d'une non-convergence. Après correction ($\text{LR}=0.1$, 500 époques), la MSE atteint ≈ 0 et les prédictions correspondent aux valeurs réelles. Ce résultat parfait est spécifique aux données synthétiques linéaires sans bruit. Sur des données réelles, une MSE nulle serait le signe d'un surapprentissage nécessitant régularisation et validation externe pour garantir la généralisation du modèle.

"""

`print(rapport)`